

Reading Assignment 9

1. The main issue in Listing 4.1 is the unsafe use of the `gets()` function. Since `gets()` doesn't check how much data it reads, an attacker can input more data than the buffer can hold. This causes a buffer overflow, letting the attacker overwrite adjacent memory, such as the function's return address. Once the return address is overwritten, the attacker gains control over the program's execution flow, making the program vulnerable to exploitation.
2. To put malicious code (or shellcode) into the running program's memory, simply embed the shellcode directly into the input. Since the vulnerable program places user input directly onto the stack without checking its size, the shellcode is copied into the program's stack memory. Essentially, I take advantage of the buffer overflow vulnerability by crafting the input so that it includes the malicious code, making it readily available in the program's memory space.
3. After inserting the malicious shellcode into the stack, we overwrite the return address stored on the stack with an address pointing back to the start of my shellcode. When the vulnerable function finishes execution and attempts to return, it will use the overwritten return address and jump directly to my malicious code instead of returning normally. This trick redirects program execution to run the attacker-controlled shellcode.
4. Address Randomization (ASLR) randomly changes memory locations to make exploitation harder. It can be disabled by running:
`sudo sysctl -w kernel.randomize_va_space=0` in the terminal.
5. To disable Stack-Guard and make the stack executable, compile using:
`gcc -fno-stack-protector -z execstack -o vulnerable vulnerable.c`.